

Отчёт по созданию динамической библиотеки для NX10.
Дисциплина: основы автоматизированного проектирования.
Алексеев Павел, группа 9, курс 3, ФКН, ВГУ.

1. Объявление стандартных функций и констант

Ниже представлен набор констант и функций, которые используются для построения деталей.

1.1. Вспомогательные Функции

1.1.1. Построение примитивов (geometry.cpp)

CreateArc строит дугу, используя библиотечную функцию UF_CURVE_create_arc_thru_3pts, строящая дуги через 3 точки, число 1 означает дугу 90 градусов.

```
37 int CreateArc(  
38     double p1[3],  
39     double p2[3],  
40     double p3[3],  
41     tag_t&    arcTag)  
42 {  
43     return UF_CURVE_create_arc_thru_3pts(  
44         1,  
45         p1, p2, p3,  
46         &arcTag);  
47 }
```

Рисунок 1 – Дуга

CreateCircle строит окружность с центром в (cx, cy, z) радиусом radius. Ее плоскость будет перпендикулярна оси 0Z. Вычисляются точки, и так же, как и в CreateArc используется UF_CURVE_create_arc_thru_3pts, но с числом 2, означающим замкнутую дугу (полную окружность).

```

4  int CreateCircle( You, 2 days ago - feat(Task3): define help funcs
1  double cx,
2  double cy,
3  double z,
4  double radius,
5  tag_t& circleTag)
6  {
7  double point1[3] = { [0]=cx + radius, [1]=cy, [2]=z };
8  double point2[3] = { [0]=cx - radius, [1]=cy, [2]=z };
9  double point3[3] = { [0]=cx, [1]=cy + radius, [2]=z };
10
11  int errorCode = UF_CURVE_create_arc_thru_3pts(
12      2,
13      point1,
14      point2,
15      point3,
16      &circleTag);
17
18  return errorCode;
19  }

```

Рисунок 2 – Окружность

CreateLine строит отрезок между двумя точками. Использует UF_CURVE_create_line.

```

21 int CreateLine(
22     const double start[3],
23     const double end[3],
24     tag_t& lineTag)
25 {
26     UF_CURVE_line_t lineData;
27     lineData.start_point[0] = start[0];
28     lineData.start_point[1] = start[1];
29     lineData.start_point[2] = start[2];
30     lineData.end_point[0] = end[0];
31     lineData.end_point[1] = end[1];
32     lineData.end_point[2] = end[2];
33
34     return UF_CURVE_create_line(&lineData, &lineTag);
35 }

```

Рисунок 3 – Линия

1.1.2. Работа с эскизами (sketch.cpp)

CreateSketchOnPlane создает эскиз на плоскости XY со смещением по оси 0Z. Использована в большинстве случаев.

```
4 int CreateSketchOnPlane(  
5     const char* name,  
6     double      zOffset,  
7     tag_t&      sketchTag)  
8 {  
9     int errorCode = 0;  
10  
11     char sketchName[129];  
12     strcpy_s(sketchName, sizeof(sketchName), name);  
13  
14     errorCode = UF_SKET_initialize_sketch(sketchName, &sketchTag);  
15     if (errorCode != 0)  
16         return errorCode;  
17  
18     double matrix[9] =  
19     {  
20         [0]=1.0, [1]=0.0, [2]=0.0,  
21         [3]=0.0, [4]=1.0, [5]=0.0,  
22         [6]=0.0, [7]=0.0, [8]=zOffset  
23     };  
24  
25     int option      = 2;  
26     tag_t obj[2]     = { NULL_TAG, NULL_TAG };  
27     int ref[2]       = { [0]=0, [1]=0 };  
28     int planeDir     = 1;  
29  
30     errorCode = UF_SKET_create_sketch(  
31         sketchName,  
32         option,  
33         matrix,  
34         obj,  
35         ref,  
36         planeDir,  
37         &sketchTag);  
38  
39     return errorCode;  
40 }
```

Рисунок 4 – Эскиз на XY

CreateSketchOnPlaneZX делает то же, что и предыдущая, только на плоскости ZX. Использована для прodelывания отверстия в патрубке.

```
43 int CreateSketchOnPlaneZX( You, 2 days ago - feat(Task3): korpus patrubok
1  const char* name,
2  tag_t& sketchTag)
3  {
4      int errorCode = 0;
5
6      char sketchName[129];
7      strcpy_s(sketchName, sizeof(sketchName), name);
8
9      errorCode = UF_SKET_initialize_sketch(sketchName, &sketchTag);
10     if (errorCode != 0)
11         return errorCode;
12
13     double matrix[9] =
14     {
15         [0]=1.0, [1]=0.0, [2]=0.0,
16         [3]=0.0, [4]=0.0, [5]=1.0,
17         [6]=0.0, [7]=0.0, [8]=0.0
18     };
19
20     int option = 2;
21     tag_t obj[2] = { NULL_TAG, NULL_TAG };
22     int ref[2] = { [0]=0, [1]=0 };
23     int planeDir = 1;
24
25     errorCode = UF_SKET_create_sketch(
26         sketchName,
27         option,
28         matrix,
29         obj,
30         ref,
31         planeDir,
32         &sketchTag);
33
34     return errorCode;
35 }
```

Рисунок 5 – Эскиз на ZX

CreateSketchOnPlaneParallel создает эскиз на плоскости, параллельной ZX со смещением по оси OY. Использована при построении бобышки.

```
80 int CreateSketchOnPlaneParallelZX( You, 1 day ago - feat(Task3): bobyshka with colysion
1  const char* name,
2  double yOffset,
3  tag_t& sketchTag)
4  {
5      int errorCode = 0;
6
7      char sketchName[129];
8      strcpy_s(sketchName, sizeof(sketchName), name);
9
10     errorCode = UF_SKET_initialize_sketch(sketchName, &sketchTag);
11     if (errorCode != 0)
12         return errorCode;
13
14     double matrix[9] =
15     {
16         [0]=1.0, [1]=0.0, [2]=0.0,
17         [3]=0.0, [4]=0.0, [5]=1.0,
18         [6]=0.0, [7]=yOffset, [8]=0.0
19     };
20
21     int option = 2;
22     tag_t obj[2] = { NULL_TAG, NULL_TAG };
23     int ref[2] = { [0]=0, [1]=0 };
24     int planeDir = 1;
25
26     errorCode = UF_SKET_create_sketch(
27         sketchName,
28         option,
29         matrix,
30         obj,
31         ref,
32         planeDir,
33         &sketchTag);
34
35     return errorCode;
36 }
```

Рисунок 6 – Эскиз на параллельной ZX

AddObjectToSketch добавляет кривые в активный эскиз через UF_SKET_add_objects.

```
118 int AddObjectsToSketch( You, 2 days ago - feat(Task3): define help funcs
1  tag_t sketchTag,
2  int count,
3  tag_t objects[])
4  {
5      return UF_SKET_add_objects(sketchTag, count, objects);
6  }
```

Рисунок 7 – Добавление кривых к эскизу

1.1.3. Работа с объемом (features.cpp)

CreateExtrusion создает вытягивание с нулевой конусностью из эскиза по начальному и конечному значениям, в указанном направлении, передавая NX способ взаимодействия с другими объектами (вычитание, объединение и т.д.). Для этого формируется список объектов, далее вызывается UF_MODL_create_extruded с указанными параметрами, создающий вытягивание. После UF_MODL_delete_list очищает память задействованную списком.

```
4 int CreateExtrusion( You, 2 days ago - feat(Task3): define help funcs
1 tag_t      sketchTag,
2 const char* startVal,
3 const char* endVal,
4 const double direction[3],
5 UF_FEATURE_SIGN sign)
6 {
7     int errorCode = 0;
8
9     uf_list_p_t objectList = nullptr;
10    errorCode = UF_MODL_create_list(&objectList);
11    if (errorCode != 0)
12        return errorCode;
13
14    errorCode = UF_MODL_put_list_item(objectList, sketchTag);
15    if (errorCode != 0)
16    {
17        UF_MODL_delete_list(&objectList);
18        return errorCode;
19    }
20
21    char startStr[15];
22    char endStr[15];
23    strcpy_s(startStr, sizeof(startStr), startVal);
24    strcpy_s(endStr, sizeof(endStr), endVal);
25
26    char* limits[2] = { [0]=startStr, [1]=endStr };
27    char taper[5] = "0.0";
28    double point[3] = { [0]=0.0, [1]=0.0, [2]=0.0 };
29    double dir[3] = { [0]=direction[0], [1]=direction[1], [2]=direction[2] };
30
31    uf_list_p_t featList = nullptr;
32
33    errorCode = UF_MODL_create_extruded(
34        objectList,
35        taper,
36        limits,
37        point,
38        dir,
39        sign,
40        &featList);
41
42    if (featList != nullptr)
43    {
44        UF_MODL_delete_list(&featList);
45    }
46    UF_MODL_delete_list(&objectList);
47
48    return errorCode;
```

Рисунок 8 – Вытягивание

В файле build.cpp присутствует вспомогательная функция для частой операции, создающая 4 отверстия в теле на плоскости Z. Внутри для каждого отверстия создается эскиз, строится окружность и выдавливается в нужном направлении с флагом UF_NEGATIVE (вычитание).

```
3 static int BuildHoles(  
4     const double positions[4][2],  
5     const char*  names[4],  
6     double      zPlane,  
7     double      radius,  
8     const char*  limit,  
9     const double dir[3])  
10 {  
11     for (int i = 0; i < 4; ++i)  
12     {  
13         tag_t holeSketch = NULL_TAG;  
14         tag_t holeCircle = NULL_TAG;  
15  
16         int errorCode = CreateSketchOnPlane(names[i], zPlane, holeSketch);  
17         if (errorCode != 0) return errorCode;  
18  
19         errorCode = CreateCircle(  
20             positions[i][0], positions[i][1],  
21             zPlane, radius, holeCircle);  
22         if (errorCode != 0) return errorCode;  
23  
24         errorCode = AddObjectsToSketch(holeSketch, 1, &holeCircle);  
25         if (errorCode != 0) return errorCode;  
26  
27         errorCode = CreateExtrusion(holeSketch, "0", limit, dir, UF_NEGATIVE);  
28         if (errorCode != 0) return errorCode;  
29     }  
30     return 0;  
31 }
```

Рисунок 9 – Создание 4 отверстий

1.2. Константы и размеры

Константы определены в файле build.cpp, представляют собой набор из переиспользуемых размеров.

```
10 static const double Z_PROKLADKA1 = 119.0; You, 1 day ago - refactor(Task3): extract part building into functions
1 static const double R_PROKLADKA1_OUTER = 70.0;
2 static const double R_PROKLADKA1_INNER = 25.0;
3 static const double Z_PROKLADKA1_TOP = 123.0;
4 static const double R_ZAGLUSHKA = R_PROKLADKA1_OUTER;
5 static const double Z_ZAGLUSHKA_TOP = 139.0;
6 static const double R_HOLE = 6.5;
7 static const double R_PITCH = 50.0;
8 static const double R_ZAGLUSHKA_BORE = 27.5;
9 static const double KP_HOLE_R = 17.5;
10 static const double KP_HOLE_Z = 59.0;
11
12 static const double Z_ORIGIN = 0.0;
13 static const double PLATE_HALF = 80.0;
14 static const double PLATE_RADIUS = 20.0;
15 static const double TUBE_OUTER_R = 30.0;
16 static const double TUBE_INNER_R = R_PROKLADKA1_INNER;
17 static const double TUBE_HEIGHT = 105.0;
18 static const double CORNER_HOLE_R = 7.5;
19 static const double CORNER_HOLE_POS = 62.5;
20 static const double PLATE_L = PLATE_HALF - PLATE_RADIUS;
21 static const double PLATE_S = PLATE_RADIUS * 0.7071067811865476;
22 static const char TUBE_HEIGHT_STR[] = "105";
23
24 static const double PITCH_POS[4][2] = {
25     [0]={ [0]=R_PITCH, [1]=0.0 },
26     [1]={ [0]=0.0, [1]=R_PITCH },
27     [2]={ [0]=-R_PITCH, [1]=0.0 },
28     [3]={ [0]=0.0, [1]=-R_PITCH }
29 };
30
31 double g_bobLength = 30.0;
32
33 static const double BOB_LENGTHS[] = { [0]=20.0, [1]=30.0, [2]=40.0 };
34 static const int NUM_BOB_LENGTHS = sizeof(BOB_LENGTHS) / sizeof(BOB_LENGTHS[0]);
```

Рисунок 10 – Константы

Так же в этом файле определена функция для определения динамически параметризуемого параметра – размера выступа бобышки от касательной плоскости патрубка в корпусе. Это заготовленный набор размеров, пользователь выбирает из стандартных.

```
46 void SetBobLength(int index) You, 23 hours ago - feat(Task3): parametrization of bobyshka
1 {
2     if (index < 0) index = 0;
3     if (index ≥ NUM_BOB_LENGTHS) index = NUM_BOB_LENGTHS - 1;
4     g_bobLength = BOB_LENGTHS[index];
5 }
```

Рисунок 11 – Параметризация

2. Создание детали «Фланец»

Процесс создания этой детали охватывает практически все вариации функций, поэтому он будет описан наиболее подробно. Сначала подготавливаются размеры и объекты.

```
524 int BuildFlanetz(void) You, 1 day ago - refactor(Task3): extract part building into functions
1 {
2     const double Z_FLANETZ      = -5.0;
3     const double Z_FLANETZ_BOTTOM = -30.0;
4     const double FL_CENTER_R    = 40.0;
5
6     tag_t flSketch = NULL_TAG;
7     tag_t flCurves[8] = { NULL_TAG, NULL_TAG, NULL_TAG, NULL_TAG,
8                           NULL_TAG, NULL_TAG, NULL_TAG, NULL_TAG };
9
10    int errorCode = CreateSketchOnPlane("Sketch_Flanetz", Z_FLANETZ, flSketch);
11    if (errorCode != 0) return errorCode;
12
13    double topLeft0[3]   = { [0]=-PLATE_L, [1]=PLATE_HALF, [2]=Z_FLANETZ };
14    double topRight0[3]  = { [0]=PLATE_L, [1]=PLATE_HALF, [2]=Z_FLANETZ };
15    double rightTop0[3]  = { [0]=PLATE_HALF, [1]=PLATE_L, [2]=Z_FLANETZ };
16    double rightBot0[3]  = { [0]=PLATE_HALF, [1]=-PLATE_L, [2]=Z_FLANETZ };
17    double botRight0[3]  = { [0]=PLATE_L, [1]=-PLATE_HALF, [2]=Z_FLANETZ };
18    double botLeft0[3]   = { [0]=-PLATE_L, [1]=-PLATE_HALF, [2]=Z_FLANETZ };
19    double leftBot0[3]   = { [0]=-PLATE_HALF, [1]=-PLATE_L, [2]=Z_FLANETZ };
20    double leftTop0[3]   = { [0]=-PLATE_HALF, [1]=PLATE_L, [2]=Z_FLANETZ };
21
22    double arcTRmid[3]   = { [0]=PLATE_L + PLATE_S, [1]=PLATE_L + PLATE_S, [2]=Z_FLANETZ };
23    double arcBRmid[3]   = { [0]=PLATE_L + PLATE_S, [1]=-PLATE_L - PLATE_S, [2]=Z_FLANETZ };
24    double arcBLmid[3]   = { [0]=-PLATE_L - PLATE_S, [1]=-PLATE_L - PLATE_S, [2]=Z_FLANETZ };
25    double arcTLmid[3]   = { [0]=-PLATE_L - PLATE_S, [1]=PLATE_L + PLATE_S, [2]=Z_FLANETZ };
```

Рисунок 12 – Подготовка для построения фланца

Далее создаются 4 линии, которые соединяются 4-мя дугами (скругления краев R=40). Эскиз выдавливается вниз на 25мм с флагом UF_NULLSIGN (без булевых операций).

```

551 | errorCode = CreateLine(topLeft0, topRight0, fLCurves[0]); You, 1 day ago - refactor(Task3): extract part building into functions
1 | if (errorCode != 0) return errorCode;
2 | errorCode = CreateArc(topRight0, arcTRmid, rightTop0, fLCurves[1]);
3 | if (errorCode != 0) return errorCode;
4 | errorCode = CreateLine(rightTop0, rightBot0, fLCurves[2]);
5 | if (errorCode != 0) return errorCode;
6 | errorCode = CreateArc(rightBot0, arcBRmid, botRight0, fLCurves[3]);
7 | if (errorCode != 0) return errorCode;
8 | errorCode = CreateLine(botRight0, botLeft0, fLCurves[4]);
9 | if (errorCode != 0) return errorCode;
10 | errorCode = CreateArc(botLeft0, arcBLmid, leftBot0, fLCurves[5]);
11 | if (errorCode != 0) return errorCode;
12 | errorCode = CreateLine(leftBot0, leftTop0, fLCurves[6]);
13 | if (errorCode != 0) return errorCode;
14 | errorCode = CreateArc(leftTop0, arcTLmid, topLeft0, fLCurves[7]);
15 | if (errorCode != 0) return errorCode;
16 |
17 | errorCode = AddObjectsToSketch(fLSketch, 8, fLCurves);
18 | if (errorCode != 0) return errorCode;
19 |
20 | {
21 |     double dir[3] = { [0]=0.0, [1]=0.0, [2]=-1.0 };
22 |     errorCode = CreateExtrusion(fLSketch, "0", "25", dir, UF_NULLSIGN);
23 |     if (errorCode != 0) return errorCode;
24 | }

```

Рисунок 13 – Пластина

После этого размечаются и строятся отверстия по углам. И вниз на 30мм вытягивается ответная часть корпуса: окружность R=40.

```

577 | { You, 1 day ago - refactor(Task3): extract part building into functions
1 |     const double fHolePos[4][2] = {
2 |         [0]={ [0]=CORNER_HOLE_POS, [1]=CORNER_HOLE_POS },
3 |         [1]={ [0]=-CORNER_HOLE_POS, [1]=CORNER_HOLE_POS },
4 |         [2]={ [0]=-CORNER_HOLE_POS, [1]=-CORNER_HOLE_POS },
5 |         [3]={ [0]=CORNER_HOLE_POS, [1]=-CORNER_HOLE_POS }
6 |     };
7 |     const char* fHoleNames[4] = {
8 |         [0]="Sketch_FL_Hole_0", [1]="Sketch_FL_Hole_1",
9 |         [2]="Sketch_FL_Hole_2", [3]="Sketch_FL_Hole_3"
10 |     };
11 |     double dir[3] = { [0]=0.0, [1]=0.0, [2]=-1.0 };
12 |     errorCode = BuildHoles(positions: fHolePos, names: fHoleNames, zPlane: Z_FLANETZ, radius: CORNER_HOLE_R, limit: "25", dir);
13 |     if (errorCode != 0) return errorCode;
14 | }
15 |
16 | {
17 |     tag_t fLCenterSketch = NULL_TAG;
18 |     tag_t fLCenterCircle = NULL_TAG;
19 |
20 |     errorCode = CreateSketchOnPlane("Sketch_FL_Center", Z_FLANETZ_BOTTOM, fLCenterSketch);
21 |     if (errorCode != 0) return errorCode;
22 |
23 |     errorCode = CreateCircle(0.0, 0.0, Z_FLANETZ_BOTTOM, FL_CENTER_R, fLCenterCircle);
24 |     if (errorCode != 0) return errorCode;
25 |
26 |     errorCode = AddObjectsToSketch(fLCenterSketch, 1, &fLCenterCircle);
27 |     if (errorCode != 0) return errorCode;
28 |
29 |     double dir[3] = { [0]=0.0, [1]=0.0, [2]=-1.0 };
30 |     errorCode = CreateExtrusion(fLCenterSketch, "0", "30", dir, UF_POSITIVE);
31 |     if (errorCode != 0) return errorCode;
32 | }

```

Рисунок 14 – Отверстия в пластине

В конце пробивается сквозное отверстие R=50.6 на 55мм.

```
611 { You, 1 day ago - refactor(Task3): extract part building into functions
1   tag_t flThroughSketch = NULL_TAG;
2   tag_t flThroughCircle = NULL_TAG;
3
4   errorCode = CreateSketchOnPlane("Sketch_FL_Through", Z_FLANETZ, flThroughSketch);
5   if (errorCode != 0) return errorCode;
6
7   errorCode = CreateCircle(0.0, 0.0, Z_FLANETZ, 25.3, flThroughCircle);
8   if (errorCode != 0) return errorCode;
9
10  errorCode = AddObjectsToSketch(flThroughSketch, 1, &flThroughCircle);
11  if (errorCode != 0) return errorCode;
12
13  double dir[3] = { [0]=0.0, [1]=0.0, [2]=-1.0 };
14  errorCode = CreateExtrusion(flThroughSketch, "0", "55", dir, UF_NEGATIVE);
15  if (errorCode != 0) return errorCode;
16  }
17
18  return 0;
19 }
```

Рисунок 15 – Отверстие

3. Создание детали «Прокладка 1»

Деталь представляет собой тонкий цилиндр с большим отверстием посередине и 4-мя малыми отверстиями для креплений расположенных на окружности диаметром 100мм.

4. Создание детали «Заглушка»

По построению схожа с прокладкой 1. Отличается отсутствием отверстия в середине (лишь вычитание на 4мм и другой радиус) и высотой цилиндра.

5. Создание детали «Прокладка 2»

Пластина та же, что и у фланца, но центральное отверстие другого радиуса.

6. Создание детали «Корпус»

Деталь состоит из нескольких других деталей.

6.1. Создание детали «Корпус.Фланец 1»

Деталь схожа с Прокладкой 2, отличается большей высотой и диаметром отверстия, которое сходно с диаметром патрубка.

6.2. Создание детали «Корпус.Патрубок»

Трубка с отверстием, центр которого на высоте 59мм.

6.3. Создание детали «Корпус.Фланец 2»

По построению практически не отличается от прокладки, кроме большей высоты.

6.4. Создание детали «Корпус.Бобышка»

Трубка, которая примыкает к патрубку и вставлена в специальное отверстие. Выносится на изменяемую длину наружу. Обрезка краев цилиндра внутри патрубка за счет вычитания цилиндра, представляющего собой копию внутреннего пространства патрубка.

7. Создание интерфейса

Интерфейс редактируется в специальном редакторе, в котором должен быть сгенерирован файл .dlx. В проекте файлы интерфейса расположены в папке UI_NX/. Файл .dlx не может быть перемещен, потому что запуск динамической библиотеки инициирует поиск файла разметки интерфейса именно по этому расположению.

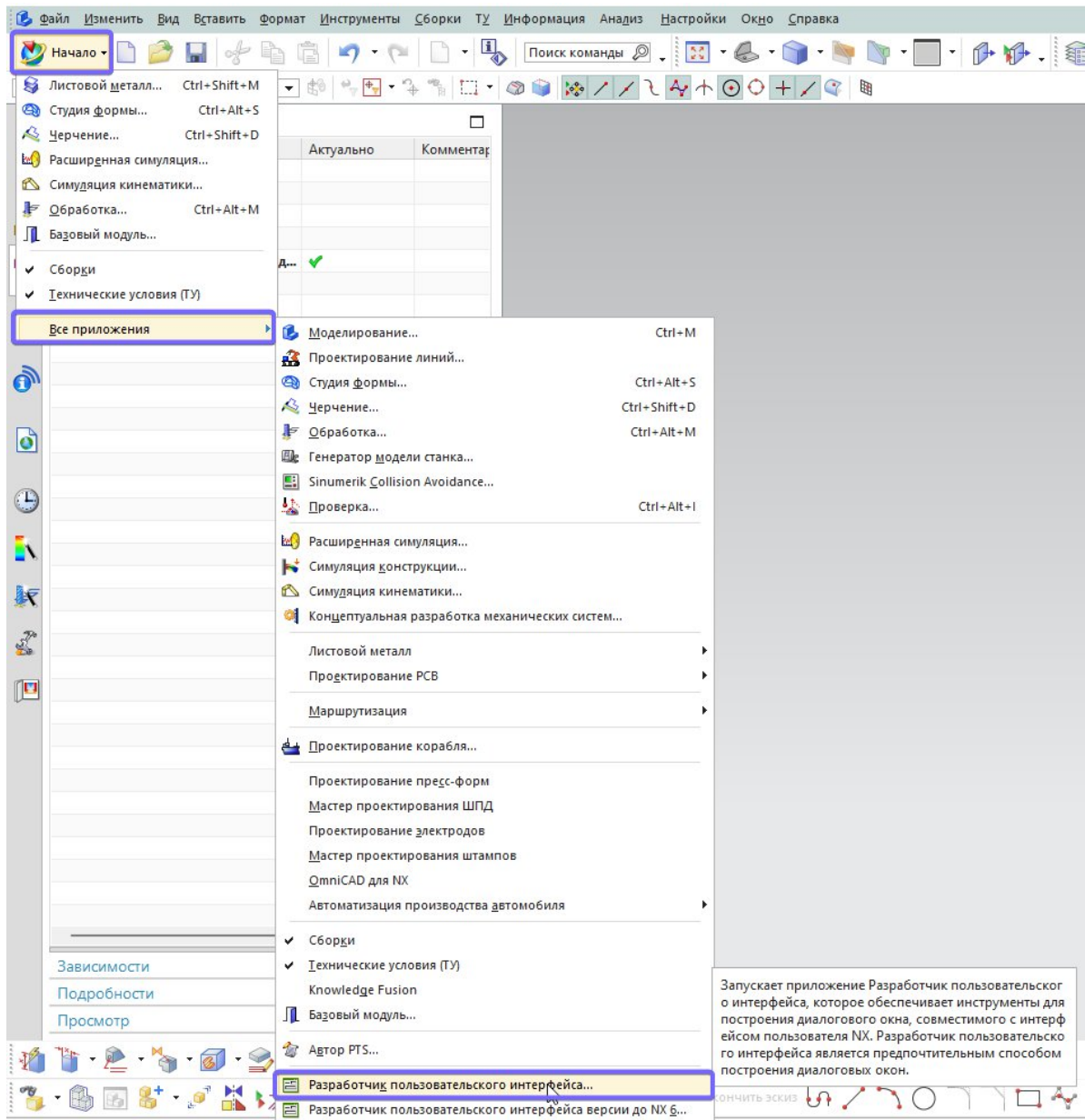


Рисунок 16 – Расположение редактора интерфейса

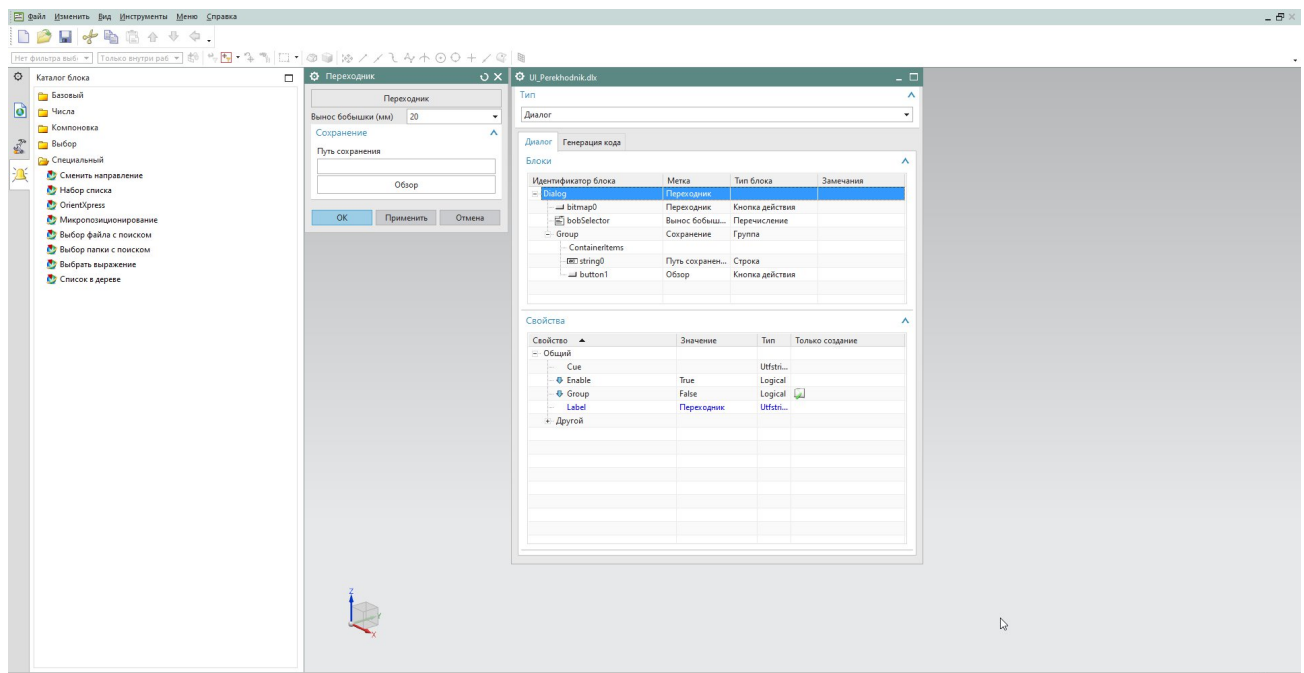


Рисунок 17 – Текущий дизайн окна

8. Запуск библиотеки

Библиотека запускается по пути x64/Release/Perekhodnik.dll. В диалоговом окне можно выбрать размер выступа бобышки и путь сохранения детали. При нажатии ОК или Применить строится деталь и сохраняется по указанному пути.

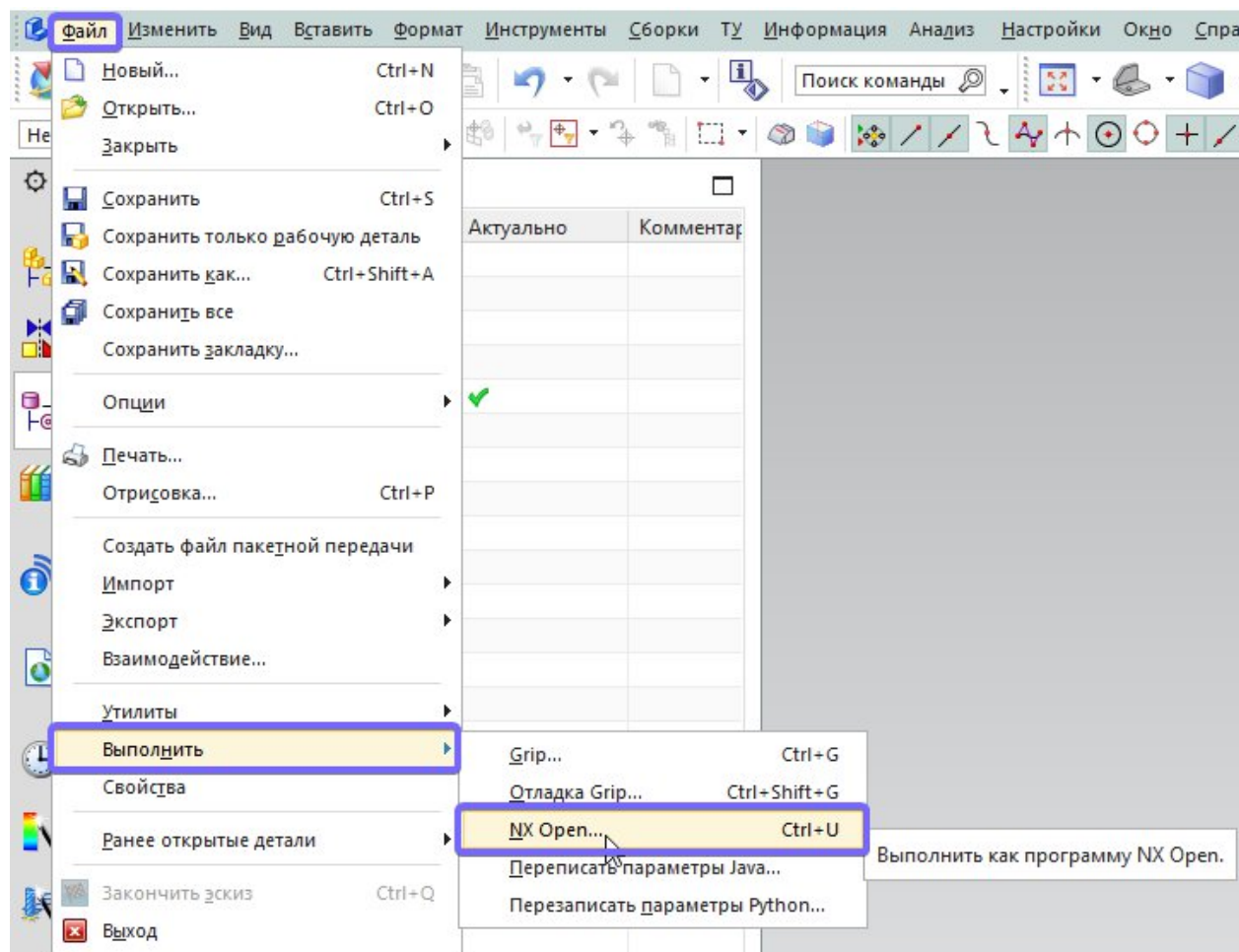


Рисунок 18 – Запуск DLL из интерфейса NX

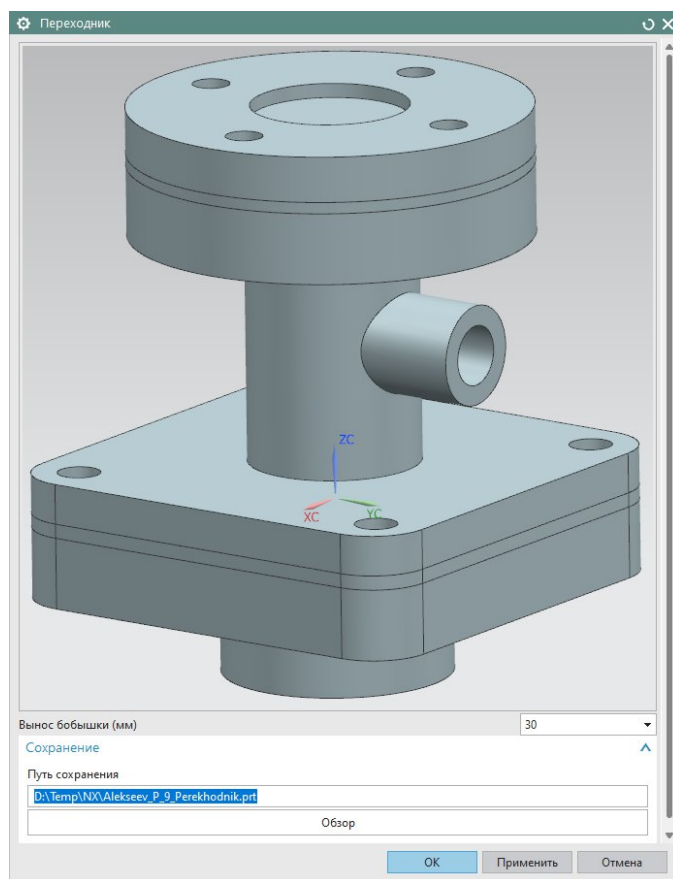


Рисунок 19 – Окно редактора построения

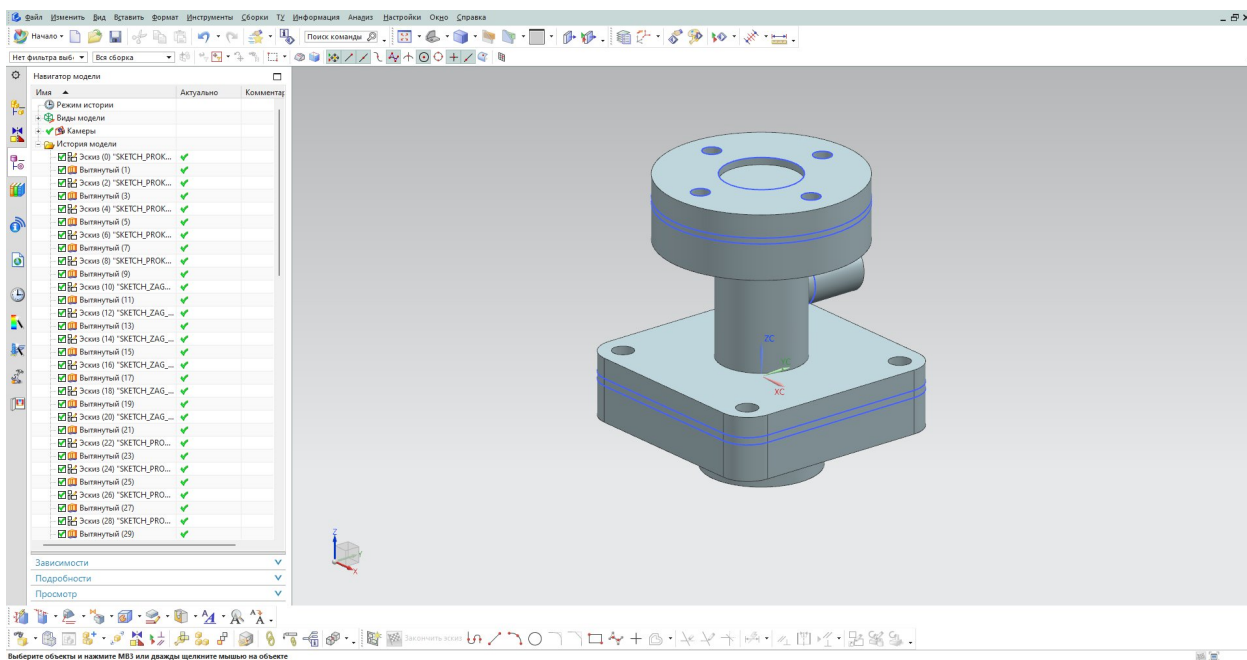


Рисунок 20 – Результат построения